Introduction to Linear Data Structures:

Definition and importance of linear data structures, Abstract data types (ADTs) and their implementation, Overview of time and space complexity analysis for linear data structures. Searching Techniques: Linear & Binary Search, Fibonacci Search, Sorting Techniques: Bubble sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort.

Q) What is data structure?

A data structure is a way of organizing, storing, and managing data in a computer so that it can be accessed and manipulated efficiently. It defines the relationship between the data and the operations that can be performed on that data.

Q) What is Linear Data Structure?

A linear data structure is a data structure in which the data elements are arranged sequentially, or linearly. Each element has a previous and next adjacent, except for the first and last elements.

Linear data structures are characterized by having a single path between any two elements. These structures are straightforward and easy to understand.

Q) What are the common examples of linear data structures?

Arrays: An array is a collection of elements stored at contiguous memory locations, where each element can be accessed directly using its index.

Linked Lists: A linked list is a collection of nodes, where each node contains data and a reference (or pointer) to the next node in the sequence.

Stacks: A stack is a collection of elements that follows the Last In, First Out (LIFO) principle, meaning that the last element added to the stack is the first one to be removed.

Queues: A queue is a collection of elements that follows the First In, First Out (FIFO) principle, meaning that the first element added to the queue is the first one to be removed.

Q) What is the Importance of linear data structures?

1. **Efficient data access.** Linear data structures, such as arrays and linked lists, allow for efficient access to individual elements by their position or index. This is because the elements are stored in a contiguous block of memory, which makes it easy to calculate the address of any given element.

2. Dynamic sizing : Linear data structures can dynamically adjust their size as elements are added or removed. This is in contrast to static data structures, such as stacks and queues, which have a fixed size.

3. Ease of implementatin: Linear data structures are relatively simple to implement and understand, making them ideal for a wide range of applications.

4. Versatility: Linear data structures can be used in various applications, such as searching, sorting, and manipulation of data. For example, arrays can be used to store a list of items, while linked lists can be used to implement a queue or a stack

5. Simple algorithms: Many algorithms used in linear data structures are simple and straightforward. This makes them easy to understand and implement, even for beginners.

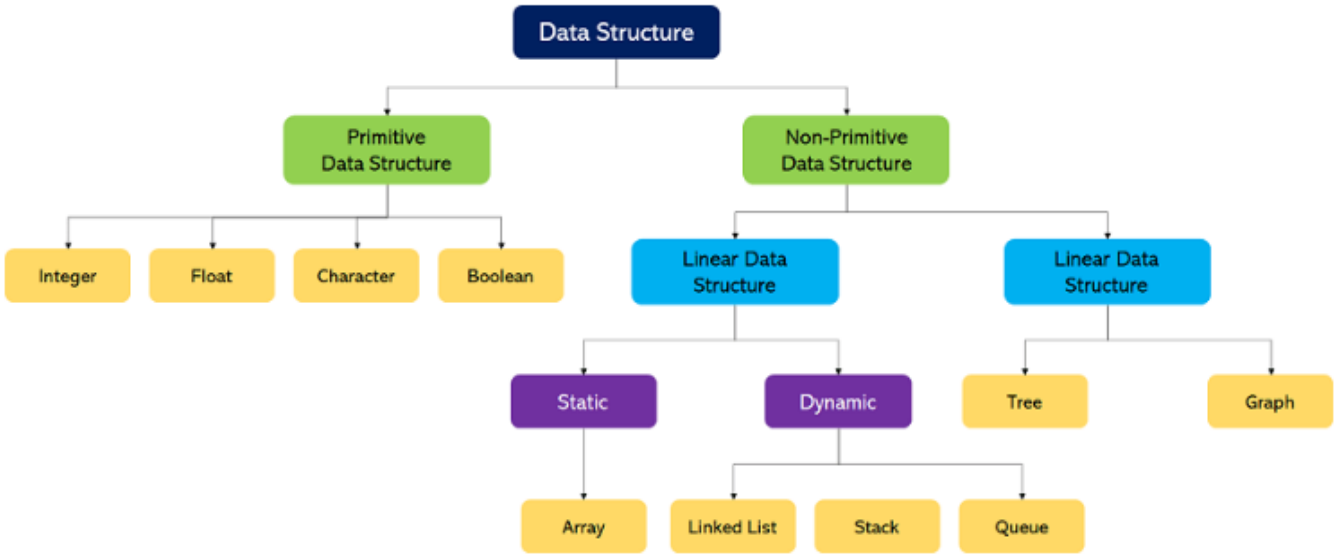
6. Optimized Operations: Linear data structures often lend themselves to optimized operations. For example, stacks and queues support efficient push and pop operations, making them suitable for implementing algorithms like depth-first search (DFS) and breadth-first search (BFS).

7. Simplicity in Traversal: Traversing linear data structures is typically straightforward, involving iterating through elements one by one. This simplicity makes linear structures suitable for tasks like searching, sorting, and processing data sequentially.

8. Memory Efficiency: Linear data structures can be memory-efficient, especially when compared to non-linear data structures like trees or graphs. Arrays, for example, allocate contiguous memory blocks, reducing memory overhead compared to linked structures that require additional pointers.

>>Classification of data structure:

The data structure can be classified into two categories namely - primitive data structure and non-primitive data structure.



Primitive Data Structures:

1. **Primitive Data Structures** are the data structures consisting of the numbers and the characters that come **in-built** into programs.
2. These data structures can be manipulated or operated directly by machine-level instructions.

3. Basic data types like **Integer, Float, Character**, and **Boolean** come under the Primitive Data Structures.
4. These data types are also called **Simple data types**, as they contain characters that can't be divided further

Non-Primitive Data Structures:

1. **Non-Primitive Data Structures** are those data structures derived from Primitive Data Structures.
2. These data structures can't be manipulated or operated directly by machine-level instructions.
3. The focus of these data structures is on forming a set of data elements that is either **homogeneous** (same data type) or **heterogeneous** (different data types).
4. Based on the structure and arrangement of data, we can divide these data structures into two sub-categories -

✓ Linear Data Structures

✓ Non-Linear Data Structures

Linear Data Structures:

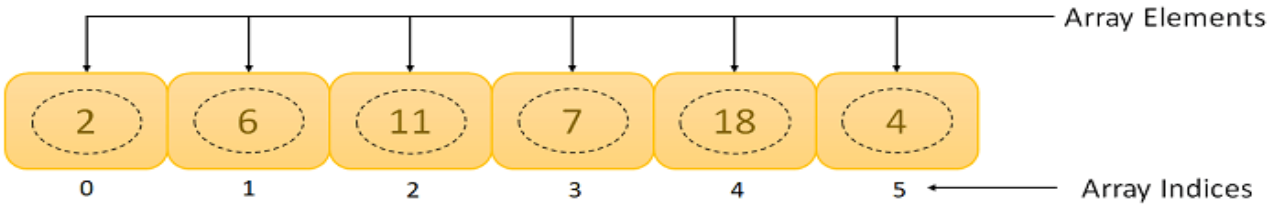
A data structure that preserves a linear connection among its data elements is known as a Linear Data Structure. The arrangement of the data is done linearly, where each element consists of the successors and predecessors except the first and the last data element.

Based on memory allocation, the Linear Data Structures are further classified into two types.

Static Data Structures: The data structures having a fixed size are known as Static Data Structures.

Example: Array

An **Array** is a data structure used to collect multiple data elements of the same data type into one variable.



Arrays can be classified into different types:

One-Dimensional Array: An Array with only one row of data elements is known as a One-Dimensional Array. It is stored in ascending storage location.

Two-Dimensional Array: An Array consisting of multiple rows and columns of data elements is called a Two-Dimensional Array. It is also known as a Matrix.

Multidimensional Array: We can define Multidimensional Array as an Array of Arrays. Multidimensional Arrays are not bounded to two indices or two dimensions as they can include as many indices are per the need.

Some Applications of Array:

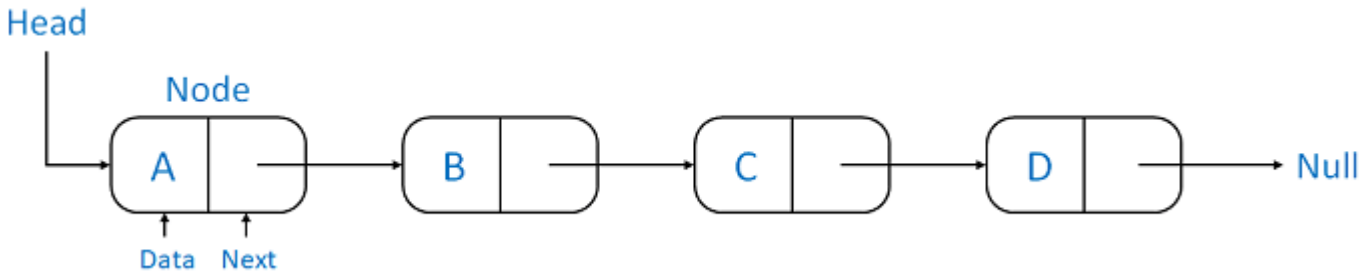
- ✓ We can store a list of data elements belonging to the same data type.
- ✓ Array acts as an auxiliary storage for other data structures.
- ✓ The array also helps store data elements of a binary tree of the fixed count.
- ✓ Array also acts as a storage of matrices.

.**Dynamic Data Structures:** The data structures having a dynamic size are known as Dynamic Data Structures.

Example: Linked Lists, Stacks, and Queues

A **Linked List** is another example of a linear data structure used to store a collection of data elements dynamically.

Data elements in this data structure are represented by the Nodes, connected using links or pointers. Each node contains two fields, the information field consists of the actual data, and the pointer field consists of the address of the subsequent nodes in the list. The pointer of the last node of the linked list consists of a null pointer, as it points to nothing.



Linked Lists can be classified into different types:

Singly Linked List: A Singly Linked List is the most common type of Linked List. Each node has data and a pointer field containing an address to the next node.

Doubly Linked List: A Doubly Linked List consists of an information field and two pointer fields. The information field contains the data. The first pointer field contains an address of the previous node, whereas another pointer field contains a reference to the next node. Thus, we can go in both directions (backward as well as forward).

Circular Linked List: The Circular Linked List is similar to the Singly Linked List. The only key difference is that the last node contains the address of the first node, forming a circular loop in the Circular Linked List.

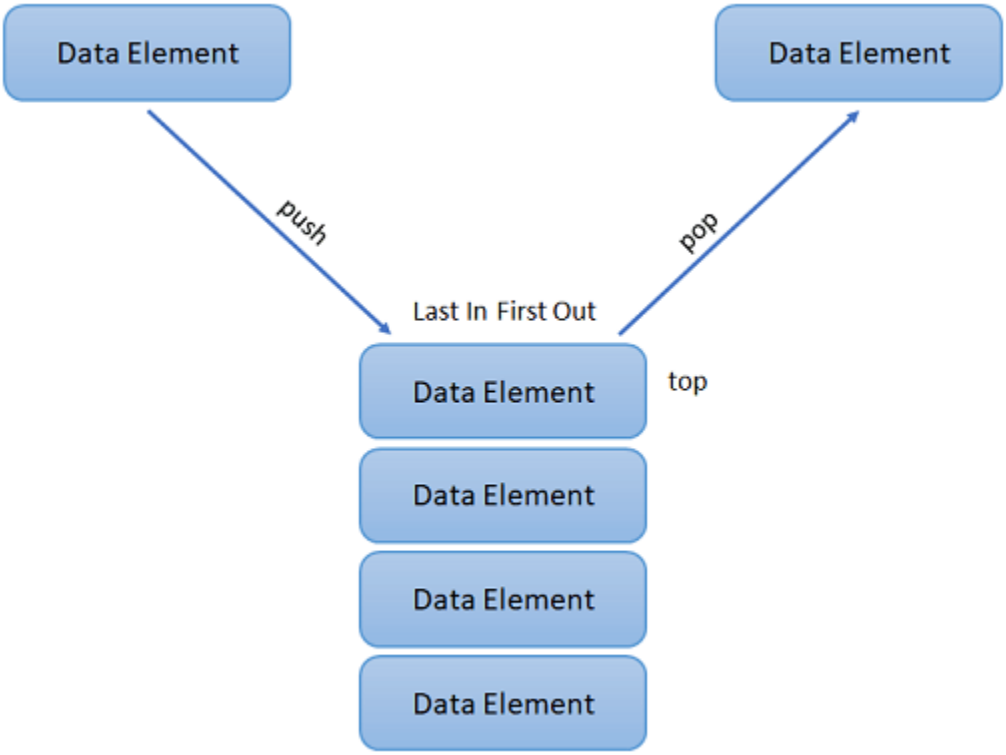
A **Stack** is a Linear Data Structure that follows the **LIFO** (Last In, First Out) principle that allows operations like insertion and deletion from one end of the Stack, i.e., Top. Stacks can be implemented with the help of contiguous memory, an Array, and non-contiguous memory, a Linked List. Real-life examples of Stacks are piles of books, a deck of cards, piles of money, and many more.

The above figure represents the real-life example of a Stack where the operations are performed from one end only, like the insertion and removal of new books from the top of the Stack.

The primary operations in the Stack are as follows:

Push: Operation to insert a new element in the Stack is termed as Push Operation.

Pop: Operation to remove or delete elements from the Stack is termed as Pop Operation.



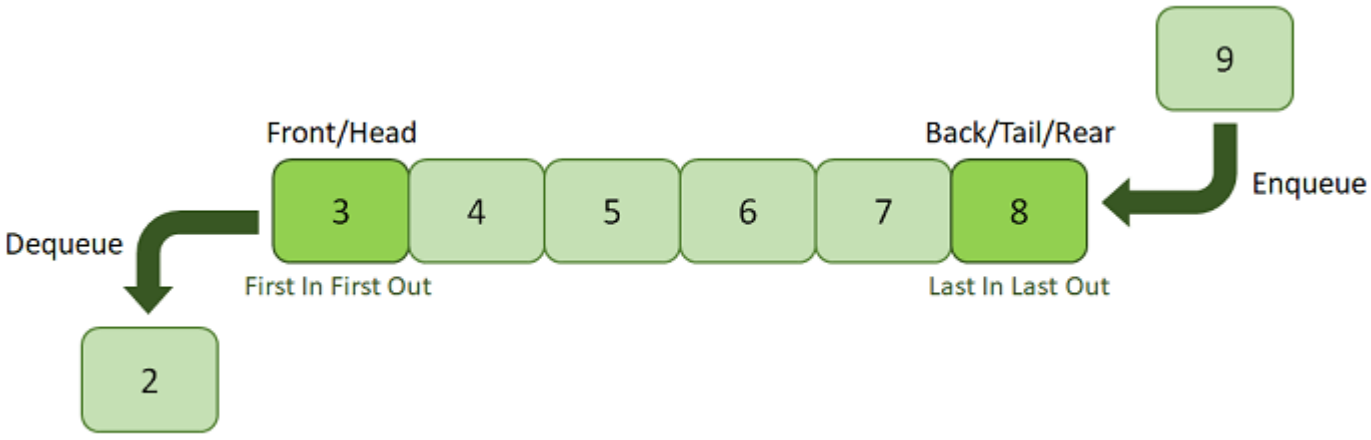
A **Queue** is a linear data structure similar to a Stack with some limitations on the insertion and deletion of the elements. The insertion of an element in a Queue is done at one end, and the removal is done at another or opposite end. Thus, we can conclude that the Queue data structure follows FIFO (First In, First Out) principle to manipulate the data elements.

The above image is a real-life illustration of a movie ticket counter that can help us understand the Queue where the customer who comes first is always served first.

The following are the primary operations of the Queue:

Enqueue: The insertion or Addition of some data elements to the Queue is called Enqueue. The element insertion is always done with the help of the rear pointer.

Dequeue: Deleting or removing data elements from the Queue is termed Dequeue. The deletion of the element is always done with the help of the front pointer.



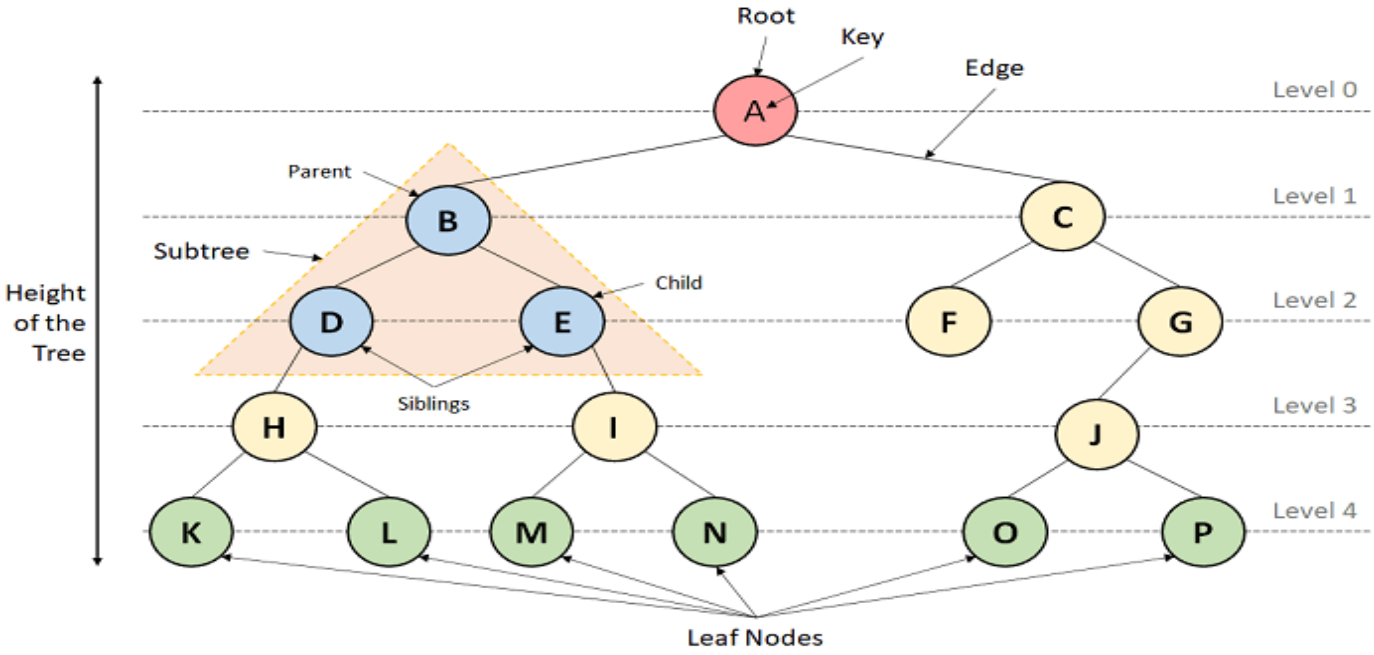
Non-Linear Data Structures:

Non-Linear Data Structures are data structures where the data elements are not arranged in sequential order. Here, the insertion and removal of data are not feasible in a linear manner. There exists a hierarchical relationship between the individual data items.

Types of Non-Linear Data Structures?

The following is the list of Non-Linear Data Structures that we generally use:

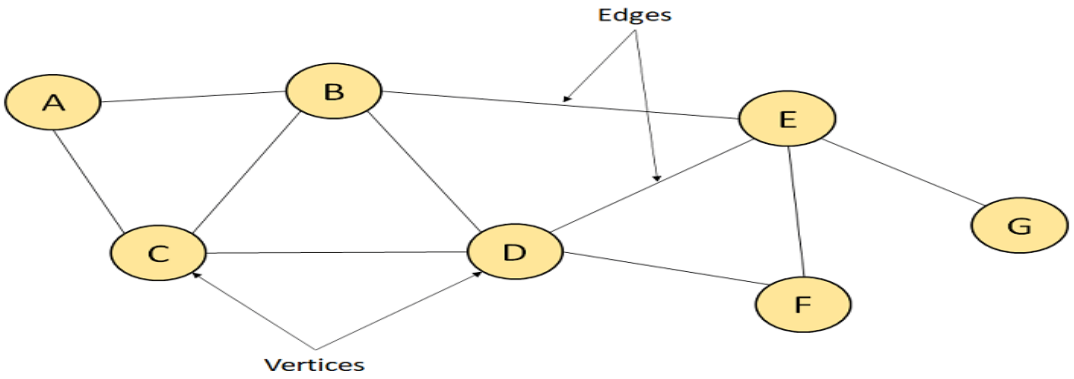
1. Trees: A Tree is a Non-Linear Data Structure and a hierarchy containing a collection of nodes such that each node of the tree stores a value and a list of references to other nodes (the "children").



2. Graphs: A Graph is another example of a Non-Linear Data Structure comprising a finite number of nodes or vertices and the edges connecting them. The Graphs are utilized to address problems of the real world in which it denotes the problem area as a network such as social networks, circuit networks, and telephone networks. For instance, the nodes or vertices of a Graph can represent a single user in a telephone network, while the edges represent the link between them via telephone.

The Graph data structure, G is considered a mathematical structure comprised of a set of vertices, V and a set of edges, E as shown below:

G = (V,E)



The above figure represents a Graph having seven vertices A, B, C, D, E, F, G, and ten edges [A, B], [A, C], [B, C], [B, D], [B, E], [C, D], [D, E], [D, F], [E, F], and [E, G].

Abstract data types (ADTs) in Data Structure and their implementation:

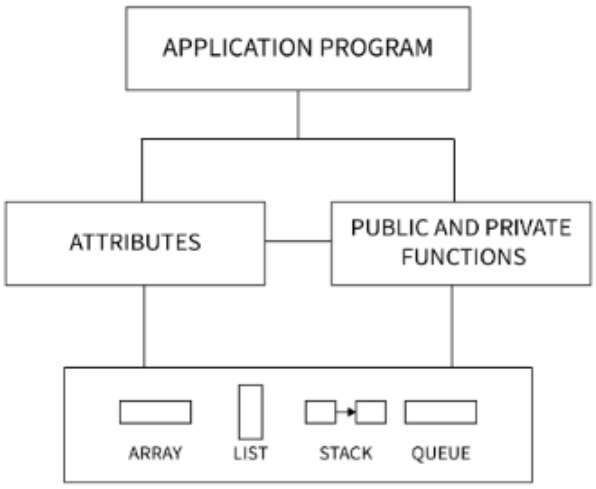
Overview: A data type defines the type of data structure. A data type can be categorized into a primitive data type (for example integer, float, double etc.) or an abstract data type (for example list, stack, queue etc.).

Q) What is Abstract Data Type in Data Structure?

An Abstract Data Type in data structure is a kind of a data type whose behavior is defined with the help of some attributes and some functions. Generally, we write these attributes and functions inside a class or a structure so that we can use an object of the class to use that particular abstract data type.

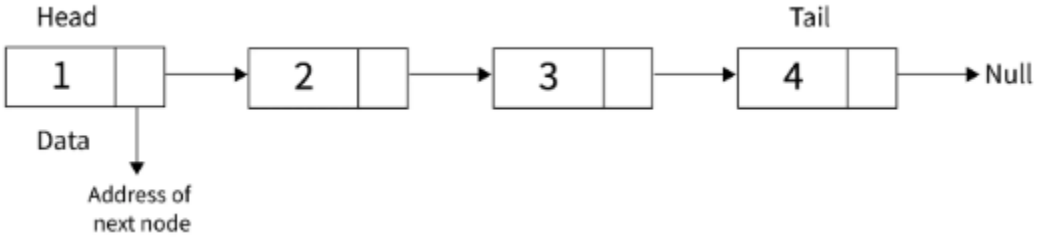
Examples of Abstract Data Type in Data Structure are list, stack, queue etc.

Abstract Data Type Model:



List ADT: Lists are linear data structures in which data is stored in a non - continuous fashion.

List consists of data storage boxes called '**nodes**'. These nodes are linked to each other i.e. each node consists of the address of some other block. In this way, all the nodes are connected to each other through these links.



Operations:

front(): returns the value of the node present at the front of the list.

back(): returns the value of the node present at the back of the list.

push_front(int val): creates a pointer with value = val and keeps this pointer to the front of the linked list.

push_back(int val): creates a pointer with value = val and keeps this pointer to the back of the linked list.

pop_front(): removes the front node from the list.

pop_back(): removes the last node from the list.

empty(): returns true if the list is empty, otherwise returns false.

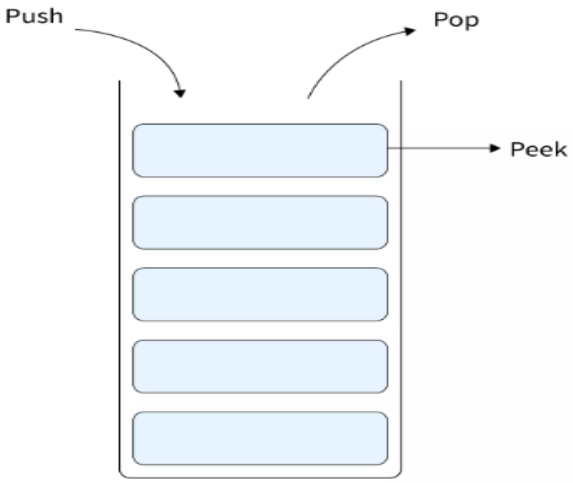
size(): returns the number of nodes that are present in the list.

Given below are some of the important operations that are defined in List ADT.

```
// defining node of the list
class node{
   public:
   int data; // to store the data
   node* next; // to store the address of the next List node
    node(int val) // a constructor to initialize the node parameters
   {
      data=val;
      next=NULL;
   }
}
class list
{
   int count=0; // to count the number of nodes in the list
   public:
   int front(); // returns value of the node present at the front of the list
   int back(); // returns value of the node present at the back of the list
   void push_front(int val); // creates a pointer with value = val and keeps this pointer to the front of the linked list
   void push_back(int val); // creates a pointer with value = val and keeps this pointer to the back of the linked list
   void pop_front(); // removes the front node from the list
   void pop_back(); // removes the last node from the list
   bool empty(); // returns true if list is empty, otherwise returns false
   int size(); // returns the number of nodes that are present in the list
};
```

Stack ADT: A stack follows the Last-In-First-Out (LIFO) principle, where the last element added to the stack is the first one to be removed.

Stack is a linear data structure in which data can be only accessed from its top. It only has two operations i.e. push (used to insert data to the stack top) and pop (used to remove data from the stack top).



Operations:

Push: Add an element to the top of the stack.

Pop: Remove and return the element from the top of the stack.

Peek (or Top): Return the element from the top of the stack without removing it.

isEmpty: Check if the stack is empty.

Size: Return the number of elements in the stack.

Given below are some of the important operations that are defined in Stack ADT:

```
class node
{
   public:
   int data; // to store data in a stack node
   node* next; // to store the address of the next node in the stack
   node(int val) // a constructor to initialize stack parameters
   {
      data=val;
      next=NULL;
   }
};
class stack(){
   int count=0; // to count number of nodes in the stack
```

```
    public:
    int top(); // returns value of the node present at the top of the stack
    void push(int val); // creates a node with value = val and put it at the stack top
    void pop(); // removes node from the top of the stack
    bool empty(); // returns true if stack is empty, otherwise returns false
    int size(); // returns the number of nodes that are present in the stack
};
```

Queue ADT: A queue follows the First-In-First-Out (FIFO) principle, where the first element added to the queue is the first one to be removed.

Queue is a linear data structure in which data can be accessed from both of its ends i.e. front and rear. It only has two operations i.e. push (used to insert data to the rear of the queue) and pop (used to remove data from the front of the queue).

Operations:

front(): returns the value of the node present at the front of the queue.

back(): returns the value of the node present at the back of the queue.

push(int val): creates a node with value = val and puts it at the front of the queue.

pop(): removes the node from the rear of the queue.

empty(): returns true if the queue is empty, otherwise returns false.

size(): returns the number of nodes that are present in the queue.

Enqueue: Add an element to the rear of the queue.

Dequeue: Remove and return the element from the front of the queue.

Given below are some of the important operations that are defined in Queue ADT.

```
class node{
    public:
    int data; // to store data in a stack node
    node* next; // to store the address of the next node in the stack
    node(int val) // a constructor to initialize stack parameters
    {
        data=val;
        next=NULL;
```

```
    }
};
class queue{
    int count=0; // to count number of nodes in the stack
    public:
    int front(); // returns value of the node present at the front of the queue
    int back(); // returns value of the node present at the back of the queue
    void push(int val); // creates a node with value = val and put it at the front of the queue
    void pop(); // removes node from the rear of the queue
    bool empty(); // returns true if queue is empty, otherwise returns false
    int size(); // returns the number of nodes that are present in the queue
};
```

Advantages of Abstract Data Type

1. Abstract data type in data structure makes it very easy for us to use the complex data structures along with their complex functions. It follows an object - oriented programming paradigm.
2. By using abstract data types, we can also customize any data structure depending on how we plan to use that particular data structure.
3. Abstract data type in data structure follows the concept of reusability of a code. This means that we don't have to write a particular piece of code again and again. We can just create an abstract data type and we can use it by simply calling the functions present in it.

Q1) what is an algorithm?

An algorithm is a step by step procedure to solve a problem.

In normal language, the algorithm is defined as a sequence of statements which are used to perform a task.

In computer science, an algorithm can be defined as follows...

-An algorithm is a sequence of unambiguous instructions used for solving a problem, which can be implemented (as a program) on a computer.

Q2) Specifications of Algorithms

Every algorithm must satisfy the following specifications...

Input - Every algorithm must take zero or more number of input values from external.

Output - Every algorithm must produce an output as result.

Definiteness - Every statement/instruction in an algorithm must be clear and unambiguous.

Finiteness - For all different cases, the algorithm must produce result within a finite number of steps.

Effectiveness - Every instruction must be basic enough to be carried out and it also must be feasible.

Q3) **Example for an Algorithm?**

Let us consider the following problem for finding the largest value in a given list of values.

Problem Statement : Find the largest number in the given list of numbers?

Input : A list of positive integer numbers. (List must contain at least one number).

Output : The largest number in the given list of positive integer numbers.

Consider the given list of numbers as 'L' (input), and the largest number as 'max' (Output).

Algorithm:

Step 1: Define a variable 'max' and initialize with '0'.
Step 2: Compare first number (say 'x') in the list 'L' with 'max', if 'x' is larger than 'max', set 'max' to 'x'.
Step 3: Repeat step 2 for all numbers in the list 'L'.
Step 4: Display the value of 'max' as a result.

Code using C Programming Language:

```
int findMax(L)
{
   int max = 0,i;
   for(i=0; i < listSize; i++)
   {
      if(L[i] > max)
          max = L[i];
   }
   return max;
}
```


Q4)Recursive Algorithm: In computer science, all algorithms are implemented with programming language functions. We can view a function as something that is invoked (called) by another function. It executes its code and then returns control to the calling function. Here, a function can be called by itself or it may call another function which in turn call the same function inside it is known as recursion.

A recursive function can be defined as follows...

-The function which is called by itself is known as Direct Recursive function (or Recursive function)

A recursive algorithm can also be defined as follows...

The function which calls a function and that function calls its called function is known Indirect Recursive function (or Recursive function).

Q5)What is Performance Analysis of an algorithm?

The formal definition is as follows...

-Performance of an algorithm is a process of making evaluative judgment about algorithms.

(OR)

-Performance of an algorithm means predicting the resources which are required to an algorithm to perform its task.

Generally, the performance of an algorithm depends on the following elements...

1. Whether that algorithm is providing the exact solution for the problem?
2. Whether it is easy to understand?
3. Whether it is easy to implement?
4. How much space (memory) it requires to solve the problem?
5. How much time it takes to solve the problem? Etc.,

When we want to analyse an algorithm, we consider only the space and time required by that particular algorithm and we ignore all the remaining elements.

Based on this information, **performance analysis of an algorithm** can also be defined as follows...

Performance analysis of an algorithm is the process of calculating space and time required by that algorithm

Performance analysis of an algorithm is performed by using the following measures..

1. Space required to complete the task of that algorithm (**Space Complexity**). It includes program space and data space
2. Time required to complete the task of that algorithm (**Time Complexity**).

Q6) what is Space complexity?

When we design an algorithm to solve a problem, it needs some computer memory to complete its execution. For any algorithm, memory is required for the following purposes...

1. To store program instructions.
2. To store constant values.
3. To store variable values.
4. And for few other things like function calls, jumping statements etc,.

Space complexity of an algorithm can be defined as follows...

Total amount of computer memory required by an algorithm to complete its execution is called as space complexity of that algorithm.

Generally, when a program is under execution it uses the computer memory for THREE reasons. They are as follows...

1. **Instruction Space:** It is the amount of memory used to store compiled version of instructions.

2. **Environmental Stack:** It is the amount of memory used to store information of partially executed functions at the time of function call.
3. **Data Space:** It is the amount of memory used to store all the variables and constants.

Note:

When we want to perform analysis of an algorithm based on its Space complexity, we consider only Data Space and ignore Instruction Space as well as Environmental Stack.

That means we calculate only the memory required to store Variables, Constants, Structures, etc.,

To calculate the space complexity, we must know the memory required to store different datatype values (according to the compiler). For example, the C Programming Language compiler requires the following...

1. 2 bytes to store Integer value.
2. 4 bytes to store Floating Point value.
3. 1 byte to store Character value.
4. 6 (OR) 8 bytes to store double value.

Consider the following piece of code.....Example-1:

```
int square(int a)
{
    return a*a;
}
```

In the above piece of code, it requires 2 bytes of memory to store variable **'a'** and another 2 bytes of memory is used for **return value.**

That means, totally it requires 4 bytes of memory to complete its execution. And this 4 bytes of memory is fixed for any input value of 'a'.

This space complexity is said to be *Constant Space Complexity*.

If any algorithm requires a fixed amount of space for all input values then that space complexity is said to be Constant Space Complexity.

Q7) What is Time complexity?

Every algorithm requires some amount of computer time to execute its instruction to perform the task. This computer time required is called time complexity.

The time complexity of an algorithm can be defined as follows...

-The time complexity of an algorithm is the total amount of time required by an algorithm to complete its execution.

Generally, the running time of an algorithm depends upon the following...

1. Whether it is running on **Single** processor machine or **Multi** processor machine.
2. Whether it is a **32 bit** machine or **64 bit** machine.

3. **Read** and **Write** speed of the machine.
4. The amount of time required by an algorithm to perform **Arithmetic** operations, logical operations, **return** value and **assignment** operations etc.,
5. **Input** data

To calculate the time complexity of an algorithm, we need to define a model machine. Let us assume a machine with following configuration...

1. It is a Single processor machine
2. It is a 32 bit Operating System machine
3. It performs sequential execution
4. It requires 1 unit of time for Arithmetic and Logical operations
5. It requires 1 unit of time for Assignment and Return value
6. It requires 1 unit of time for Read and Write operations

Now, we calculate the time complexity of following example code by using the above-defined model machine...

Consider the following piece of code...

Example-1:

```
int sum(int a, int b)
{
  return a+b;
}
```

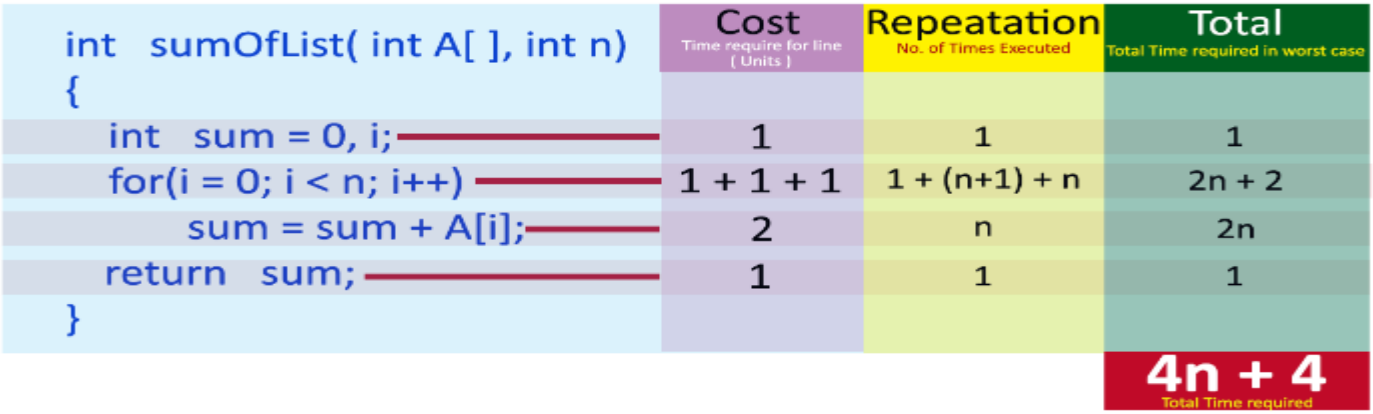
In the above sample code, it requires 1 unit of time to calculate a+b and 1 unit of time to return the value. That means, totally it takes 2 units of time to complete its execution. And it does not change based on the input values of a and b. That means for all input values, it requires the same amount of time i.e. 2 units.

If any program requires a fixed amount of time for all input values then its time complexity is said to be Constant Time Complexity.

Example-2:

```
int sum(int A[], int n)
{
  int sum = 0, i;
  for(i = 0; i < n; i++)
    sum = sum + A[i];
  return sum;
}
```

For the above code, time complexity can be calculated as follows...



In above calculation :

Cost is the amount of computer time required for a single operation in each line.

Repeatation is the amount of computer time required by each operation for all its repeatations.

Total is the amount of computer time required by each operation to execute.

So above code requires **'4n+4' Units** of computer time to complete the task. Here the exact time is not fixed. And it changes based on the n value. If we increase the n value then the time required also increases linearly.

Totally it takes '4n+4' units of time to complete its execution and it is *Linear Time Complexity*.

******If the amount of time required by an algorithm is increased with the increase of input value then that time complexity is said to be Linear Time Complexity*******

Q8) What is Asymptotic Notation?

Whenever we want to perform analysis of an algorithm, we need to calculate the complexity of that algorithm. But when we calculate the complexity of an algorithm it does not provide the exact amount of resource required. So instead of taking the exact amount of resource, we represent that complexity in a general form (Notation) which produces the basic nature of that algorithm. We use that general form (Notation) for analysis process.

- **Asymptotic notation of an algorithm is a mathematical representation of its complexity.**

Note - In asymptotic notation, when we want to represent the complexity of an algorithm, we use only the most significant terms in the complexity of that algorithm and ignore least significant terms in the complexity of that algorithm (Here complexity can be Space Complexity or Time Complexity).

For example, consider the following time complexities of two algorithms...

- **Algorithm 1 : $5n^2 + 2n + 1$**
- **Algorithm 2 : $10n^2 + 8n + 3$**

Generally, when we analyze an algorithm, we consider the time complexity for larger values of input data (i.e. **'n'** value). In above two time complexities, for larger value of **'n'** the term **'2n + 1'** in algorithm 1 has least significance than the term **'$5n^2$'**, and the term **'8n + 3'** in algorithm 2 has least significance than the term **'$10n^2$'**.

Here, for larger value of **'n'** the value of most significant terms ($\mathbf{5n^2}$ and $\mathbf{10n^2}$) is very larger than the value of least significant terms (**2n + 1** and **8n + 3**). So for larger value of **'n'** we ignore the least significant terms to represent overall time required by an algorithm. In asymptotic notation, we use only the most significant terms to represent the time complexity of an algorithm.

Majorly, we use THREE types of Asymptotic Notations and those are as follows..

1. **Big - Oh(O) :** Big-Oh notation is used to define the **upper bound** of an algorithm in terms of Time Complexity i.e. it indicates the maximum time required by an algorithm for all input values.
2. **Big - Omega (Ω):** it is used to define the **lower bound** of an algorithm in terms of Time Complexity i.e. it indicates the minimum time required by an algorithm for all input values.
3. **Big - Theta (Θ):** It is used to define the **average bound** of an algorithm in terms of Time Complexity i.e. It indicates the average time required by an algorithm for all input values.

Q9)What is Data Structure?

Whenever we want to work with a large amount of data, then organizing that data is very important. If that data is not organized effectively, it is very difficult to perform any task on that data. If it is organized effectively then any operation can be performed easily on that data.

A data structure can be defined as follows...

-**Data structure is a method of organizing a large amount of data more efficiently so that any operation on that data becomes easy.**

Note:

- ❖ Every data structure is used to organize the large amount of data
- ❖ Every data structure follows a particular principle
- ❖ The operations in data structure should not violate the basic principle of that data structure.

Based on the organizing method of data structure, data structures are divided into two types.

- **Linear Data Structures**
- **Non - Linear Data Structures**

Q10) what is Linear Data Structures?

If a data structure organizes the data in sequential order, then that data structure is called a Linear Data Structure.

Example:

1. Arrays 2.List (Linked List) 3.Stack 4.Queue

Q11) what is Non - Linear Data Structures?

- If a data structure organizes the data in random order, then that data structure is called as Non-Linear Data Structure.

Example: 1.Tree 2.Graph 3.Dictionaries 4.Heaps 5.Tries

>>Linear Search in C:

Searching is a method to find some relevant information in a data set.

Overview: The Linear Search algorithm in C **sequentially** checks each element of the list until the key element is found or the entire list has been traversed. Therefore, it is known as a sequential search. The **time complexity** of the linear search algorithm in C is **O(n)**, and space complexity is **O(1)**. It is easy to learn and understand

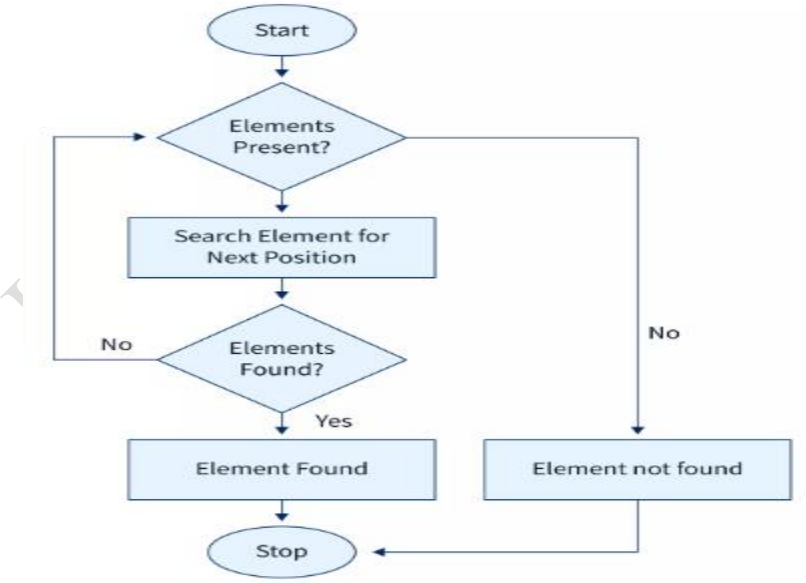
Introduction to Linear Search in C

Linear Search is the most basic method of searching an element from a list. It is also known as sequential search, as it sequentially checks each element of the list until the **key element** is found or the entire list has been traversed. It uses conditional statements and relational operators to find whether the given element is present in the **list or not**. It is easy to learn and implement.

Approach to Implement Linear Search Algorithm in C

- Take input of the element that is going to be searched. It can be referred to as a key element.
- Compare every element of the list with the key element starting from the leftmost end of the list.
- If any element of the list matches with the key, return the index of that element.
- If the entire list has been traversed and none of the elements matched with the key, then return -1, which specifies the key element is not present in the list.

Flow Chart of the Linear Search Algorithm:



Implementation of Linear Search Program in C

Below code will search the key element in the input array using linear search in C.

```
#include<stdio.h>
int main() {
  // declaration of the array and other variables
```

```
  int arr[20], size, key, i, index;
  printf("Number of elements in the list: ");
  scanf("%d", &size);
  printf("Enter elements of the list: ");
  // loop for the input of elements from 0 to number of elements-1
  for (i = 0; i < size; i++)
    scanf("%d", &arr[i]);
  printf("Enter the element to search ie. key element: ");
  scanf("%d", &key);
  // loop for traversing the array from 0 to the number of elements-1
  for (index = 0; index < size; index++)
    if (arr[index] == key) // comparing each element with the key element
      break; // cursor out of the loop when a key element found
  if (index < size) // condition to check whether previous loop partially traversed or not
    printf("Key element found at index %d", index); // printing the index if key found
  else
    printf("Key element not found");
  return 0;
}
```

Output: Number of elements in the list: 5

Enter elements of the list: 1 2 3 4 5

Enter the element to search ie. key element: 4

A key element found at index 3

Time and Space Complexity for the above code:

The time required to search an element using a linear search algorithm depends on the **size of the array** as the whole array is being traversed. In the **best-case** scenario, the **key element** is caught at the beginning of the array, and in the worst case, each element is being compared, and the last one is the key element. Therefore, The time complexity of a linear search algorithm in C is **O(n).** Space Complexity is **O(1)** as no extra space is being taken.

Linear Search in C for Multiple Occurrences

The previous code was designed because the key element occurs only once in the array. Here, the **linear search** algorithm in C is going to be modified for the multiple occurrences of the key element. The code is designed to find the number of occurrences of the key element in the array along with their positions.

```
#include<stdio.h>
```

```
int main() {
  // declaration of the array and other variables
  int arr[20], size, key, i, index;
  int countKey = 0; //initializing count of key element as 0
  printf("Number of elements in the list: ");
  scanf("%d", &size);
  printf("Enter elements of the list: ");
  // loop for the input of elements from 0 to number of elements-1
  for (i = 0; i < size; i++)
    scanf("%d", &arr[i]);
  printf("Enter the element to search ie. key element: ");
  scanf("%d", &key);
  // loop for traversing the array from 0 to the number of elements-1
  for (index = 0; index < size; index++) {
    if (arr[index] == key) { // comparing each element with the key element
      printf("Key element found at index %d\n", index); // printing the index if key found
      countKey++; // incrementing the count of key element;
    }    }
  if (countKey == 0) // condition to check whether key element is found or not
    printf("Key element not found");
  else
    printf("Key element is present %d times in the array.\n", countKey);
  return 0;  }
```

Output: Number of elements in the list: 6 Enter elements of the list: 1 2 3 2 4 2

Enter the element to search ie. key element: 2

Key element found at index 1

Key element found at index 3

Key element found at index 5

The key element is present 3 times in the array.

Time and Space Complexity for the above code:

The time required to search multiple elements using a linear search algorithm in C depends on the size of the array as the whole array is being traversed. In the best-case scenario as well as in the worst-case, each element of the array is being compared. Therefore, The time complexity of a linear search algorithm in C is O(n). Space Complexity is O(1) as no extra space is being taken.

>>C Program for Binary Search (Data Structure):

Overview

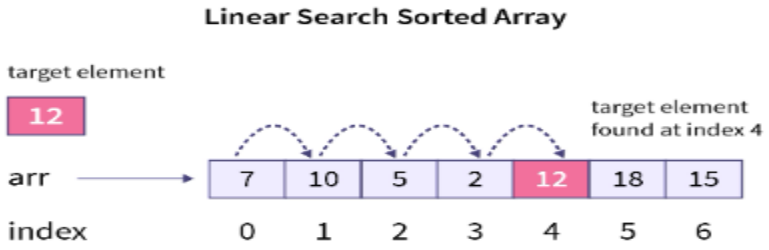
Binary Search in C is a searching algorithm, that is used to **search an element in a sorted array**. It is one of the most used and basic searching algorithm in computer science

Binary Search

Binary Search in C is generally used to solve a wide range of issues in computer science and real-world scenarios **related to searching**.

We have many search algorithms like Linear Search or Sequential Search, Binary Search, Jump Search, Fibonacci Search, etc. and one of the most efficient is the **Binary Search** algorithm.

In linear search, we compare each element with the target from the start to the end of an array with time complexity of **O(n)** , while the binary search takes only **O(log N)** time, we will see how.



Binary search algorithm **operates on a sorted array to find a target element** and it is more efficient than the linear search algorithm. So, let's see how does binary search algorithm work.

Conditions for when to apply Binary Search in a Data Structure

To use the Binary Search algorithm effectively, the following conditions must be met:

1. The data structure must be sorted.
2. Access to any element of the data structure takes constant time

Binary Search Algorithm

1. **Initialization**:
    - Set left to the index of the first element in the search space.
    - Set right to the index of the last element in the search space.
2. **Search**:
    - Calculate the middle index mid as (left + right) / 2.
    - Compare the element at index mid with the target key.
    - If the key is found at mid, the search is successful, and the process terminates.
    - If the key is not found at mid, proceed to the next step.
3. **Choose Search Space**:
    - If the element at index mid is smaller than the target key, narrow down the search space to the right half by setting left to mid + 1.

- If the element at index mid is larger than the target key, narrow down the search space to the left half by setting right to mid - 1.

4. **Repeat**:
   - Repeat steps 2 and 3 while left is less than or equal to right.
   - If the key is found, return mid.
5. **Termination**:
   - If left becomes greater than right, then the target key is not in the dataset, and the search process concludes.

How does Binary Search work?

In binary search, we reduce the search space in half at each iteration, find the mid index, and compare the middle element with the target element. If the target element is bigger than the middle element, we reduce the array to the right half only, beginning with the mid + 1 (just next to mid) index. If the target element is smaller than the middle element, we reduce the array to the left half only, ending before the mid - 1 index (just before the mid).

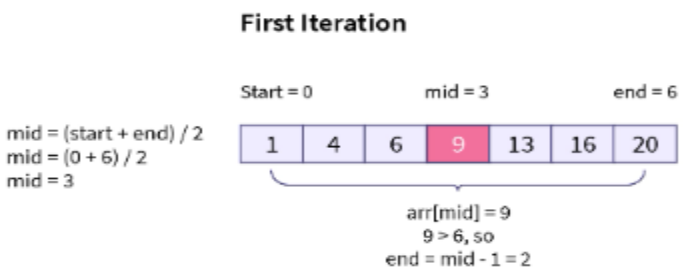
Now suppose, we are provided with a ***sorted array*** and a target element (let's say **k**) and we want to search if **k** exists in the array or not, and if **k** does exist in the array, we return the index/position of **k** in the array.

So, let's consider a sorted array shown in the image below and try to search number **6** and return its index. Please note the array has a total of **7** elements.

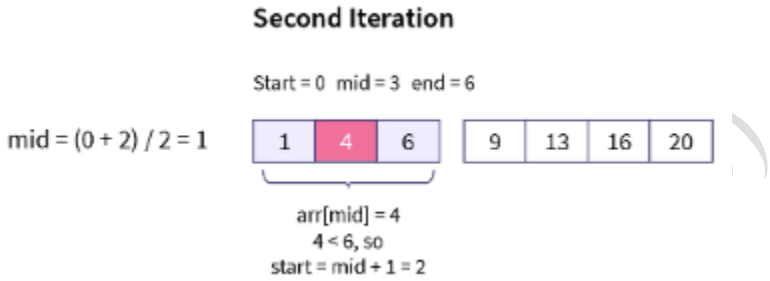
- First, Let's initialize some variables:

  start = 0; // index of first element in the array
  end = 6; // index of last element in the array
  mid = (start + end) / 2 = 3 // index of the middle element in the array.

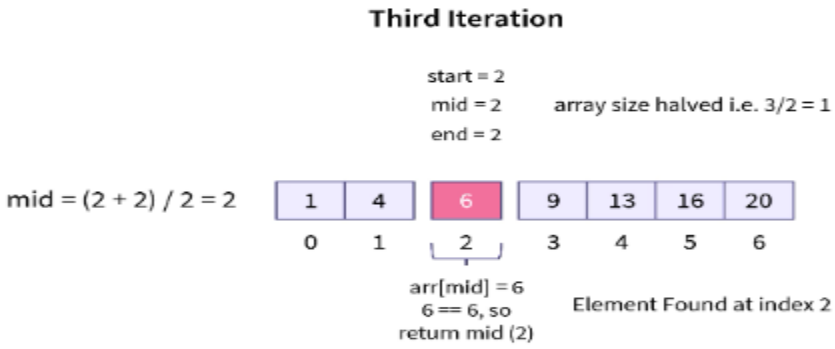
- In the **first iteration** of binary search, we check if the middle element is equal to **6**. If it is equal, we return the **mid** index. Here, **arr[mid] = arr[3] = 9**, i.e., not equal to **6**. So, we check if the middle element is greater than or less than **6**. Now, **9** is greater than **6**, so we assign **mid - 1** (**= 2**) index to the **end** variable that reduces the array size by half (we know that **6** will be present on the left side of the array, as we are working in a sorted array).



- In the **second iteration**, again, we check if the middle element is equal to, greater than, or smaller than **6**. Now, the middle element **4** is greater than **6**, so we assign **mid + 1** index to the **start** variable.



- In the **third iteration**, the middle element equals **6**, so we return the **mid** index value, i.e., **2**.



Complexity Analysis of Binary Search

The time complexity of binary search algorithm is: O(1) in the best case. O(log N) in the worst case.

Advantages : It is efficient because it continually divides the search space in half until it finds the element or only one element remains in the list to be tested.

- It specifies whether the element being searched is before or after the current place in the list.
- It is the fastest searching algorithm with the worst-case time complexity of **O(log N)** and works best for large lists.

Drawbacks of Binary Search

- The list/array must be sorted before applying the binary search algorithm. For example, with lists where elements are added constantly it is difficult to make the list remain sorted.
- It is a little more complicated as compared to linear search in searching an element from the list.
- It is not that efficient as compared to other searching algorithms for smaller list/array sizes.

FIBONACCI SEARCH:

✓ It was developed by Kiefer in 1953.

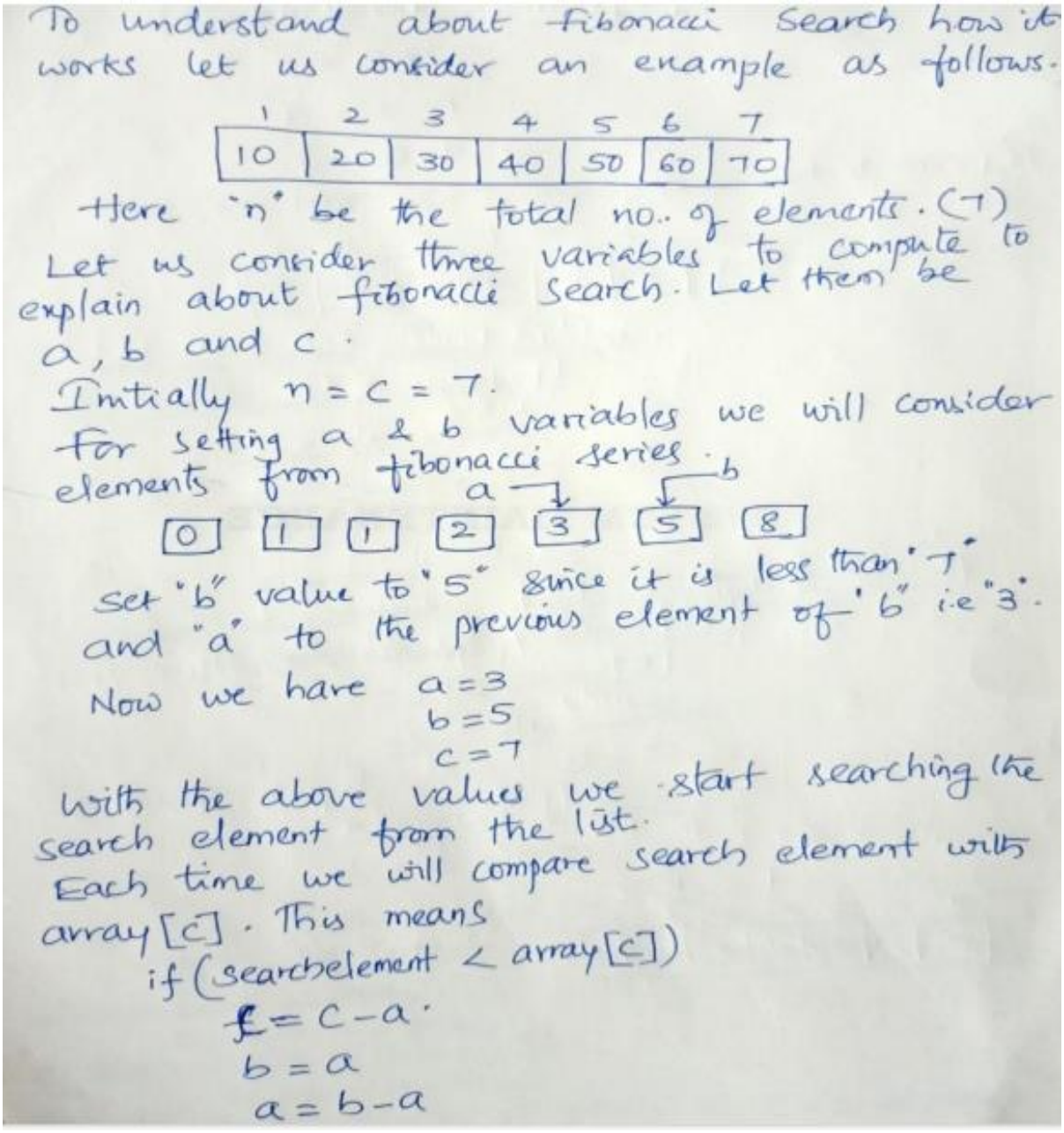
✓ In Fibonacci search we consider the indices as numbers from Fibonacci series.

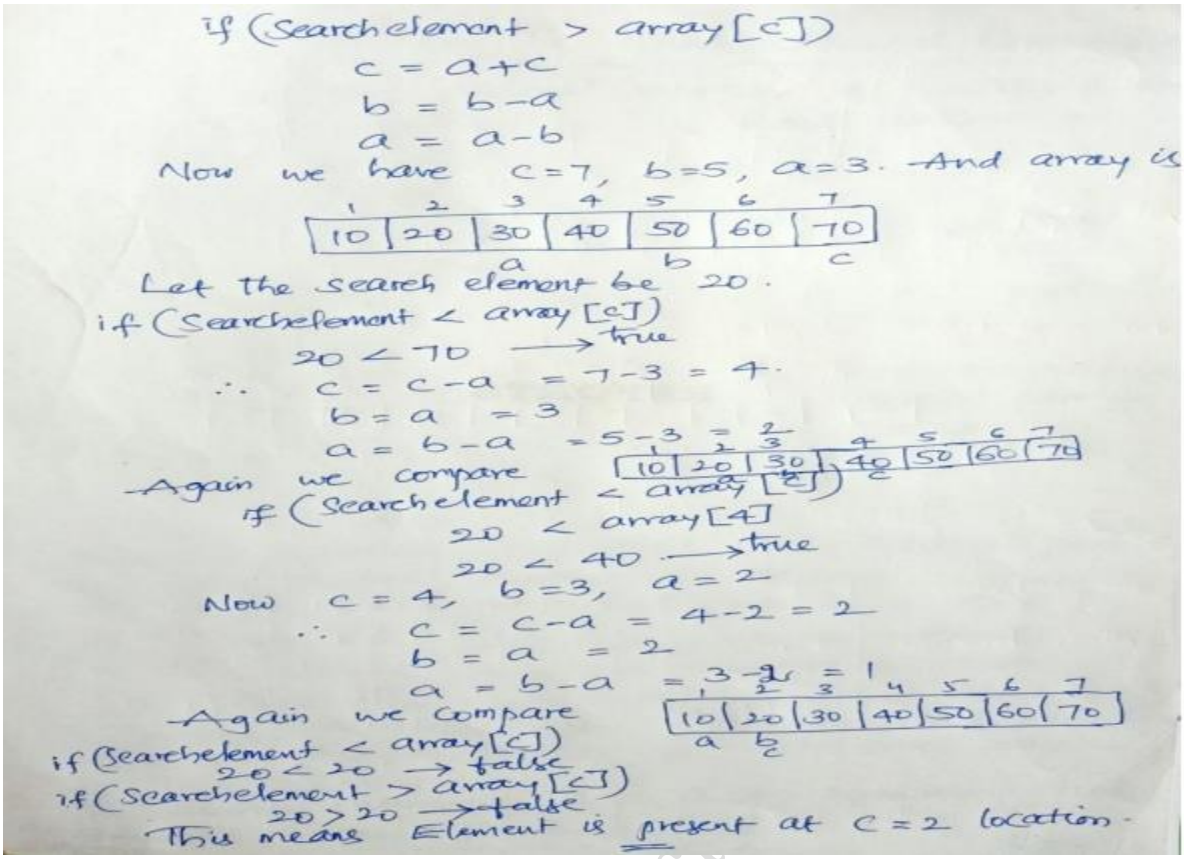
✓ To apply Fibonacci search algorithm the list that contains elements should be in sorted order.

✓ The time complexity of Fibonacci search algorithm is O(log n)

✓ It works on the principle divide - conquer strategy.

Example of Fibonacci Search:





SORTING:

DEFINITION: Sorting is a technique to rearrange the list of elements either in ascending or descending order, which can be numerical, alphabetical or any user-defined order.

Types of Sorting :

Internal Sorting:

✓ If the data to be sorted remains in main memory and also the sorting is carried out in main memory then it is called internal sorting.

The following are some internal sorting techniques:

✓ Insertion sort ✓ Merge Sort ✓ Quick Sort

External Sorting:

✓ If the data resides in secondary memory and is brought into main memory in blocks for sorting and then result is returned back to secondary memory is called external sorting.

The following are some external sorting techniques:

✓ Two-Way External Merge Sort ✓ K-way External Merge Sort

BUBBLE SORT/EXCHANGE SORT/ COMPARISON SORT :

✓ It is easiest and simple sort technique but inefficient.

✓ It is not a stable sorting technique.

✓ The time complexity of bubble sort is O(n2) in all cases.

✓ Bubble sort uses the concept of passes.

✓ The phases in which the elements are moving to acquire their proper positions is called passes.

✓ It works by comparing adjacent elements and bubbles the largest element towards right at the end of the first pass.

✓ The largest element gets sorted and placed at the end of the sorted list.

✓ This process is repeated for all pairs of elements until it moves the largest element to the end of the list in that iteration.

✓ Bubble sort consists of (n-1) passes, where n is the number of elements to be sorted.

✓ In 1st pass the largest element will be placed in the nth position.

✓ In 2nd pass the second largest element will be placed in the (n-1)th position.

✓ In (n-1)th pass only the first two elements are compared.

Algorithm for Bubble Sort:

BUBBLE_SORT(A, N)

Step 1: Repeat Step 2 For I = to N-1

Step 2: Repeat For J = to N - I

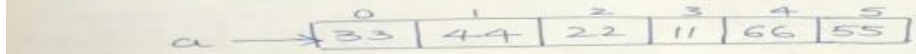
Step 3: IF A[J] > A[J + 1]

SWAP A[J] and A[J+1]

Step 4: EXIT

Example for Bubble Sort:

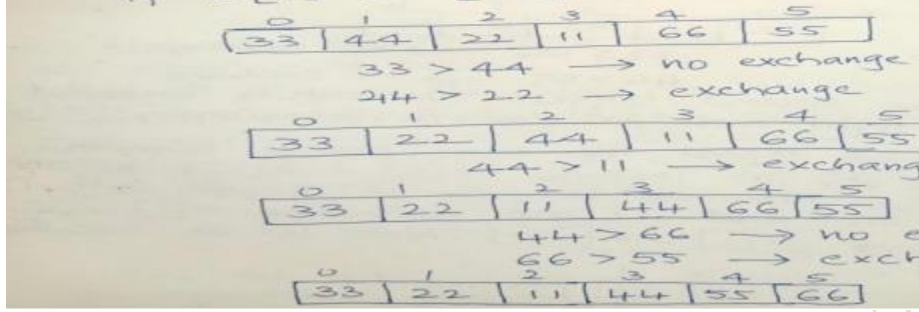
To explain about bubble sort technique, let us consider the list of elements stored in the form of an array as follows.



Now we want to sort the above array using the Bubble sort technique in ascending order.

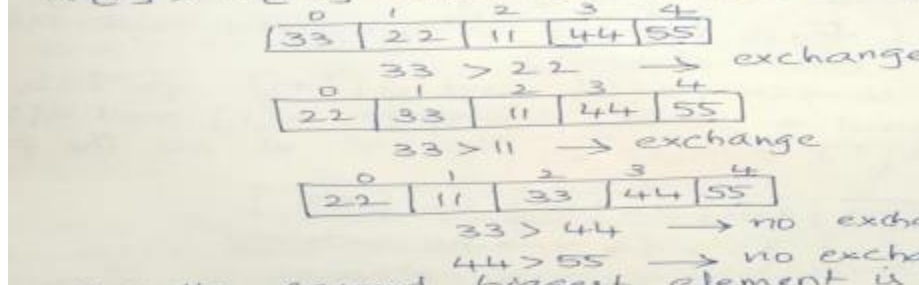
Pass 1 : (First element is compared with all other elements)

We compare a[i] and a[i+1] for i = 0, 1, 2, 3 and 4 and exchange a[i] and a[i+1] if a[i] > a[i+1]. Now let us see the process.



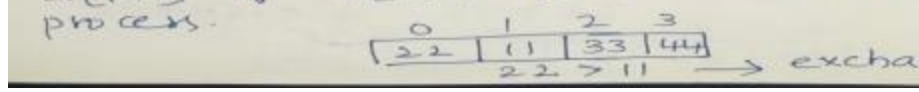
Now we can see the biggest element is bubbled to the right most position in the array. (a[5])

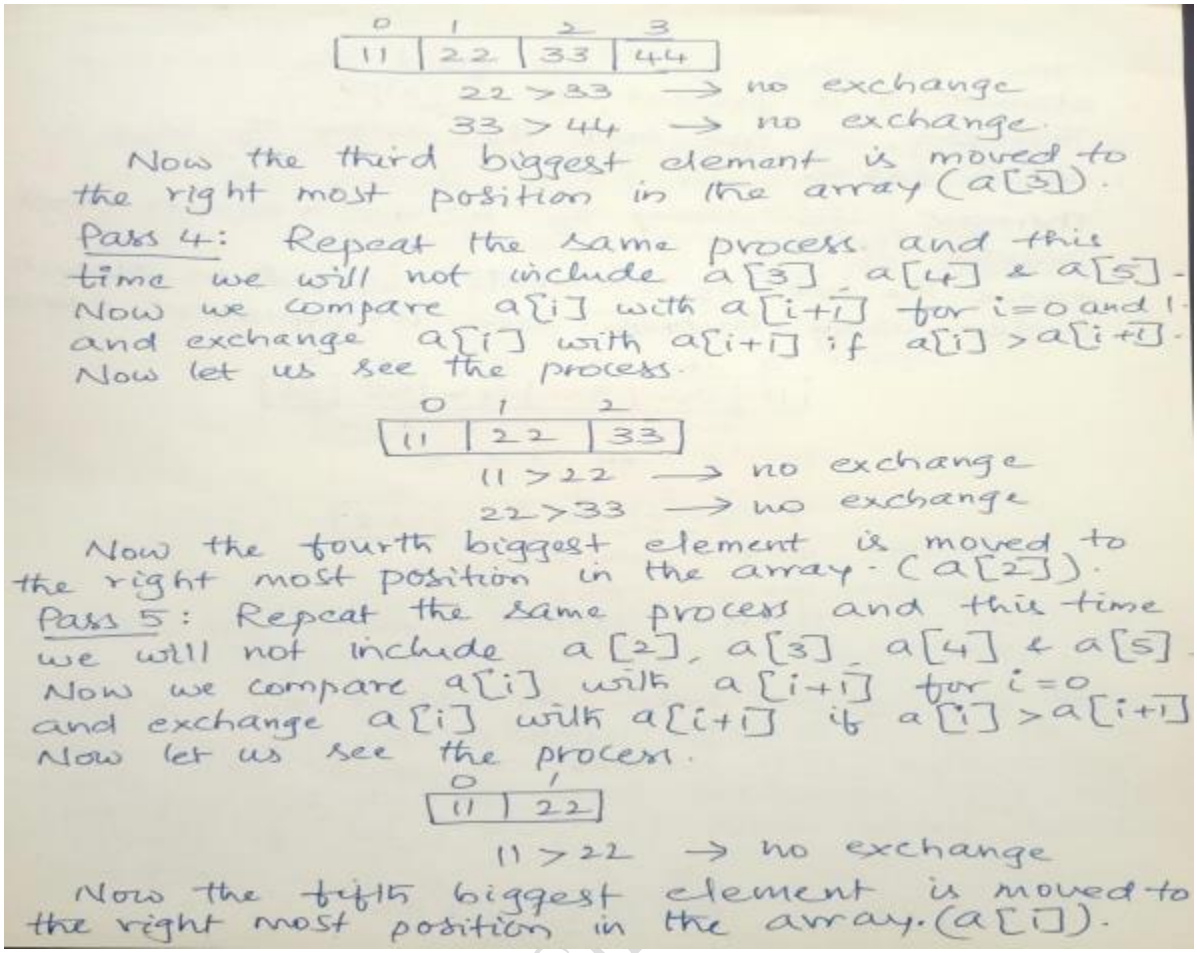
Pass 2 : We repeat the same process but we will not include a[5] in our comparison. Now we compare a[i] with a[i+1] for i = 0, 1, 2 and 3 and exchange a[i] and a[i+1] if a[i] > a[i+1]. Now let us see the process.

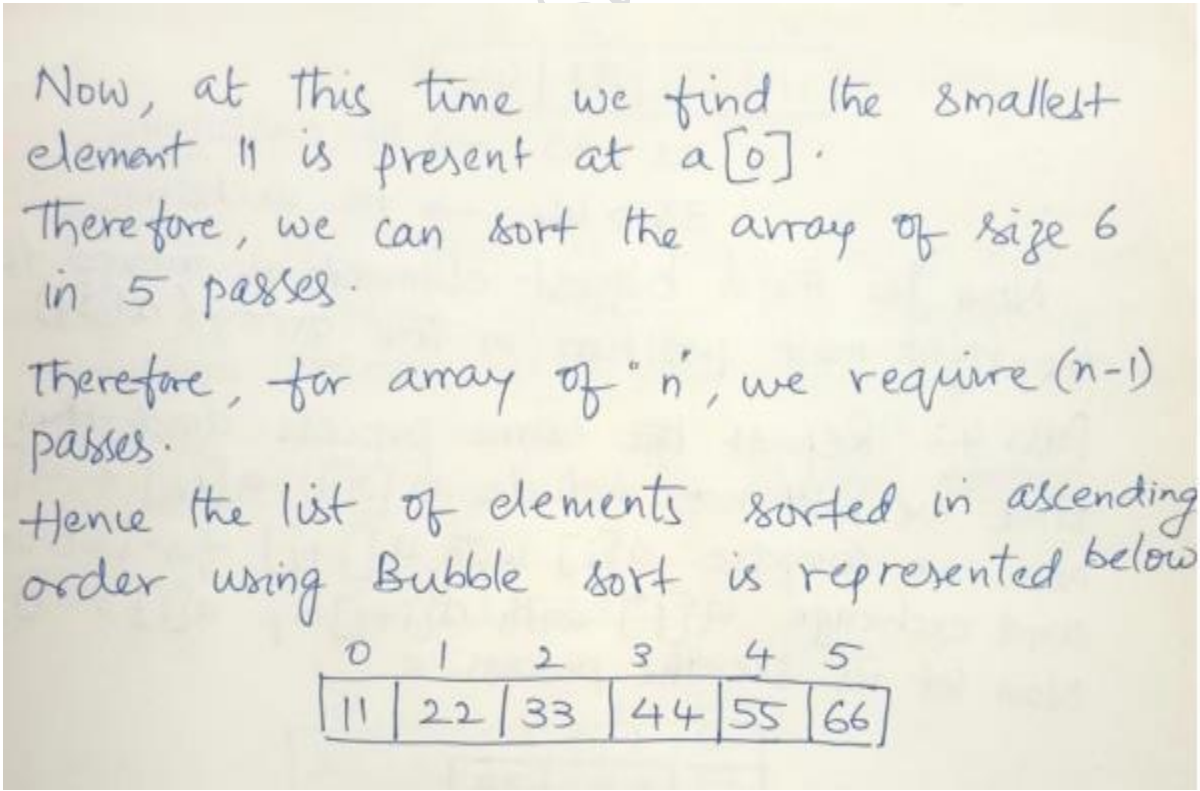


Now the second biggest element is bubbled to the right most position in the array. (a[4])

Pass 3 : We repeat the same process but this time we will not include a[5] & a[4] in our comparison. Now we compare a[i] with a[i+1] for i = 0, 1 and 2 and exchange a[i] with a[i+1] if a[i] > a[i+1]. Now let us see the process.







SELECTION SORT:

✓ It is easy and simple to implement

✓ It is used for small list of elements

✓ It uses less memory

✓ It is efficient than bubble sort technique

✓ It is not efficient when used with large list of elements It is not efficient than insertion sort technique when used with large list

✓ The time complexity of selection sort is O(n2)

✓ Consider an array A with N elements. First find the smallest element in the array and place it in the first position. Then, find the second smallest element in the array and place it in the second position. Repeat this procedure until the entire array is sorted.

✓ In Pass 1, find the position POS of the smallest element in the array and then swap A[POS] and A[0]. Thus, A[0] is sorted.

✓ In Pass 2, find the position POS of the smallest element in sub-array of N–1 elements. Swap A[POS] with A[1]. Now, A[0] and A[1] is sorted.

✓ In Pass N–1, find the position POS of the smaller of the elements A[N–2] and A[N–1]. Swap A[POS] and A[N–2] so that A[0], A[1], ..., A[N–1] is sorted.

Algorithm for Selection Sort:

SELECTION SORT(A, N)
Step 1: Start
Step 2: Repeat Steps 3 and 4 for I = 1 to N
Step 3: Call SMALLEST(A, I, N, pos)
Step 4: Swap A[I] with A[pos]
Step 5: Stop
SMALLEST (A, I, N, pos)
Step 1: Start
Step 2: SET small = A[I]
Step 3: SET POS = I
Step 4: Repeat for J = I+1 to N
 If small> A[J]
 SET small = A[J]
 SET pos = J
Step 4: Return pos
Step 5: Stop

Example for Selection Sort:

1	2	3	4	5	6	7	8	9	Remarks
65	70	75	80	50	60	55	85	45	find the first smallest element
i								j	swap a[i] & a[j]
45	70	75	80	50	60	55	85	65	find the second smallest element
	i			j					swap a[i] and a[j]
45	**50**	75	80	70	60	55	85	65	Find the third smallest element
		i				j			swap a[i] and a[j]
45	**50**	**55**	80	70	60	75	85	65	Find the fourth smallest element
			i		j				swap a[i] and a[j]
45	**50**	**55**	**60**	70	80	75	85	65	Find the fifth smallest element
				i				j	swap a[i] and a[j]
45	**50**	**55**	**60**	**65**	80	75	85	70	Find the sixth smallest element
					i			j	swap a[i] and a[j]
45	**50**	**55**	**60**	**65**	**70**	75	85	80	Find the seventh smallest element
						i j			swap a[i] and a[j]
45	**50**	**55**	**60**	**65**	**70**	**75**	85	80	Find the eighth smallest element
							i	J	swap a[i] and a[j]
45	**50**	**55**	**60**	**65**	**70**	**75**	**80**	**85**	The outer loop ends.

QUICK SORT/ PARTITION EXCHANGE SORT:

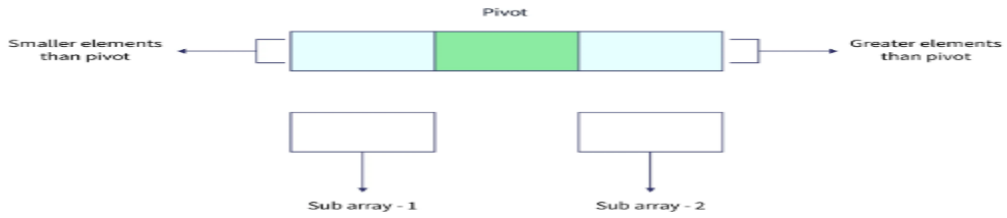
✓ It is developed by C.A.R. Hoare.

✓ This sorting algorithm uses divide and conquer strategy.

✓ In this method, the division is carried out dynamically.

✓ It contains three steps:

✓ Divide – split the array into two sub arrays so that each element in the right sub array is greater than the middle element and each element in the left sub array is less than the middle element. The splitting is done based on the middle element called **pivot**. All the elements less than pivot will be in the left sub array and all the elements greater than pivot will be on right sub array.



•Each sub array is further divided until each sub array contains only single array element. Then we will combine all such sub arrays to form a single sorted array.

✓ Conquer – recursively sort the two sub arrays.

✓ Combine – combine all the sorted elements in to a single list.

✓ Consider an array A[i] where i is ranging from 0 to n – 1 then the division of elements is as follows:

A[0]……A[m – 1], A[m], A[m + 1] …….A[n]

✓ The partition algorithm is used to arrange the elements such that all the elements are less than pivot will be on left sub array and greater then pivot will be on right sub array.

✓ The time complexity of quick sort algorithm in worst case is O(n2),

best case and average case is O(n log n).

✓ It is faster than other sorting techniques whose time complexity is O(n log n)

Algorithm for Quick Sort:

QUICK_SORT (A, LOW, HIGH)

Step 1: IF (LOW < HIGH)

CALL PARTITION (A, LOW, HIGH, MID)

CALL QUICKSORT(A, LOW, MID - 1)

CALL QUICKSORT(A, MID + 1, HIGH)

Step 2: EXIT

Algorithm for Partition:

PARTITION (A, LOW, HHIGH, MID)

Step 1: SET PIVOT = A[LOW], I =LOW, J = HIGH

Step 2: Repeat Steps 3 to 5 while I <= LOW

Step 3: Repeat while A[LOW] <= A[PIVOT]

SET I = I + 1

Step 4: Repeat while A[j] >= PIVOT

SET J = J – 1

Step 5: Repeat if I <= J

SWAP A[I], A[J]

Step 6: SWAP A[LOW], A[J]

Step 7: Return J

Step 8: EXIT

Example for Quick Sort:

Let us consider the array of elements to sort them using quick sort technique

50, 30, 10, 90, 80, 20, 40, 70

MERGE SORT:

✓ This sorting algorithm uses divide and conquer strategy.

✓ In this method, the division is carried out dynamically.

✓ It contains three steps:

✓ Divide – split the array into two sub arrays s1 and s2 with each n/2 elements. If A is an array containing zero or one element, then it is already sorted. But if there are more elements in the array, divide A into two sub-arrays, s1 and s2, each containing half of the elements of A.

✓ Conquer – sort the two sub arrays s1 and s2.

✓ Combine – combine or merge s1 and s2 elements into a unique sorted list.

✓ The time complexity of merge sort is O(n log n) in all cases.

Algorithm for Merge Sort:

MERGE_SORT(A,LOW, HIGH)

Step 1: IF LOW < HIGH

SET MID = (LOW +HIGH)/2

CALL MERGE_SORT (A, LOW, MID)

CALL MERGE_SORT (A, MID + 1, HIGH)

COMBINE (A, LOW, MID, HIGH)

Step 2: EXIT

Algorithm for Combine:

COMBINE (A, LOW, MID, HIGH)

Step 1: SET I = LOW, J = MID + 1, INDEX = LOW

Step 2: Repeat while (I <= MID) AND (J<=HIGH)

IF A[I] < A[J]

SET TEMP[INDEX] = A[I]

SET I = I + 1

SET INDEX = INDEX + 1

ELSE

SET TEMP[INDEX] = A[J]

SET J = J + 1

SET INDEX = INDEX + 1

Step 3: [Copy the remaining elements of right sub-array, if any]

IF I > MID

Repeat while J <= HIGH

SET TEMP[INDEX] = A[J]

SET J = J + 1

SET INDEX = INDEX + 1

[Copy the remaining elements of left sub-array, if any]

ELSE

IF A[I]<= MID

SET TEMP[INDEX] = A[I]

SET I = I + 1

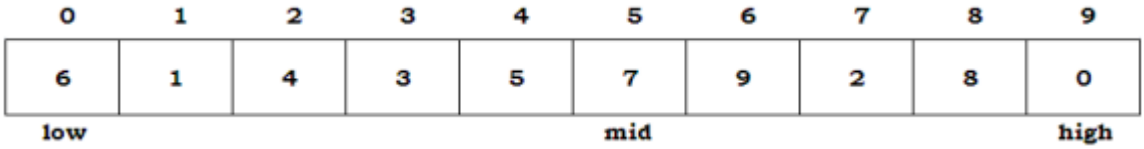
SET INDEX = INDEX + 1

Step 4: EXIT

Example for Merge Sort:

Let us consider the array of elements to sort them using Merge sort technique

6, 1, 4, 3, 5, 7, 9, 2, 8, 0



We then first make the two sublists and combine the two sorted sublists as a unique sorted list.